

Modelado de la Comunicación entre Procesos Distribuidos en un Sistema Bancario y su Validación Experimental mediante Sockets TCP y UDP en Java 21

F. Farinango, J. Gaibor y I. Villamarín

Carrera de Software, Facultad de Ciencias de la Computación y Diseño Digital Universidad Técnica Estatal de Quevedo — Asignatura: Aplicaciones Distribuidas — Quevedo, Ecuador

Resumen—Las aplicaciones distribuidas modernas dependen del intercambio correcto de mensajes entre procesos que se ejecutan en máquinas distintas. Este trabajo modela la comunicación entre procesos de un sistema bancario distribuido y la relaciona con una implementación práctica de sockets TCP y UDP desarrollada en Java 21 sobre un proyecto Maven (guía01-sockets). Se describen los procesos participantes cliente bancario, API Gateway, servicios de autenticación, cuentas, transferencias, notificaciones, auditoría, broker de mensajería y base de datos, se construye el diagrama de comunicación correspondiente y se identifica, para cada interacción, el tipo de comunicación, el protocolo y el middleware empleados junto con su justificación. Los resultados experimentales de la práctica (comandos HORA, ECO y SALIR sobre TCP, y confirmación ACK sobre UDP) validan que TCP es adecuado para operaciones críticas que exigen entrega confiable, mientras que UDP y la mensajería asíncrona resultan apropiados para eventos no críticos. Se concluye que la elección del mecanismo de comunicación debe derivarse de los requisitos de confiabilidad, latencia y desacoplamiento de cada operación.

Abstract—Modern distributed applications rely on the correct exchange of messages between processes running on different machines. This work models the inter-process communication of a distributed banking system and relates it to a practical implementation of TCP and UDP sockets developed in Java 21 on a Maven project (guía01-sockets). The participating processes banking client, API Gateway, authentication, accounts, transfers, notifications, audit, message broker and banking database— are described, the corresponding communication diagram is built, and for each interaction the communication type, protocol and middleware are identified together with their justification. The experimental results (HORA, ECO and SALIR commands over TCP, and ACK confirmation over UDP) validate that TCP is suitable for critical operations requiring reliable delivery, whereas UDP and asynchronous messaging are appropriate for non-critical events. We conclude that the choice of communication mechanism must be derived from the reliability, latency and decoupling requirements of each operation.

Palabras clave aplicaciones distribuidas, comunicación entre procesos, sockets, TCP, UDP, middleware, mensajería asíncrona, sistema bancario.

I. INTRODUCCIÓN

Una aplicación distribuida está formada por procesos que cooperan a través de una red para ofrecer un servicio común. Como esos procesos no comparten memoria, toda su coordinación descansa en el intercambio de mensajes, y la forma en que ese intercambio se realiza condiciona el rendimiento, la confiabilidad y la seguridad del sistema completo. Comprender los mecanismos de comunicación

entre procesos es, por tanto, el punto de partida para diseñar cualquier servicio distribuido serio.

En la primera semana de la asignatura se desarrolló la práctica guía01-sockets, en la que se implementó en Java 21, sobre un proyecto Maven, una comunicación cliente-servidor empleando sockets TCP y UDP. Esa práctica resolvió el problema en el plano experimental, pero un sistema real no se describe únicamente por su código: necesita un escenario de negocio que justifique por qué se eligió cada mecanismo de comunicación. El presente informe une ambos planos: conserva la evidencia técnica de la práctica y la eleva a un escenario realista, el de un sistema bancario distribuido.

El objetivo es modelar la comunicación entre los procesos de un sistema bancario, representarla en un diagrama, identificar para cada interacción su tipo, protocolo y middleware con la debida justificación, y mostrar cómo la práctica realizada en Java valida experimentalmente esas decisiones. El documento se organiza así: la Sección II describe el escenario; la III fundamenta los conceptos; la IV y la V presentan el modelado y el diagrama; la VI identifica protocolos y middleware; la VII relaciona el modelo con la práctica; las Secciones VIII a X recogen el procedimiento, los resultados y las notas de exposición; y las Secciones XI y XII discuten y concluyen.

II. DESCRIPCIÓN DEL ESCENARIO DE NEGOCIO: SISTEMA BANCARIO DISTRIBUIDO

El escenario corresponde a la plataforma digital de un banco que atiende a sus clientes a través de aplicaciones web y móviles. Las operaciones típicas son el inicio de sesión, la consulta de saldo y la transferencia de fondos entre cuentas; a ellas se suman avisos como la confirmación de una sesión o de un movimiento. Estas operaciones tienen exigencias muy distintas: el login, la consulta de saldo y la transferencia son críticas, pues un dato perdido, duplicado o desordenado tiene consecuencias económicas directas; en cambio, una notificación informativa puede perderse o llegar con retraso sin comprometer la corrección del sistema.

Para soportar alta disponibilidad, escalabilidad y mantenibilidad, la plataforma se organiza como un conjunto de servicios independientes que se comunican entre sí. Esta separación permite que cada servicio evolucione y escale por separado, y obliga a definir con cuidado cómo dialogan entre ellos: qué interacciones deben ser confiables y síncronas, y cuáles pueden ser asíncronas y desacopladas.

III. FUNDAMENTACIÓN TEÓRICA

A. Comunicación entre procesos distribuidos

La comunicación entre procesos (IPC) distribuidos consiste en el envío y recepción de mensajes entre procesos que no comparten memoria. Sus dos estilos básicos son la comunicación síncrona, en la que el emisor espera la respuesta antes de continuar, y la asíncrona, en la que el emisor prosigue sin bloquearse. La elección entre ambos estilos determina el acoplamiento temporal del sistema y es la decisión de diseño central de toda arquitectura distribuida.

B. Sockets

Los sockets son el mecanismo de IPC más difundido para construir arquitecturas cliente-servidor sobre redes TCP/IP. Un socket es un punto final de comunicación identificado por una dirección IP y un puerto, sobre el que las aplicaciones leen y escriben flujos o datagramas [1]. Por su cercanía al sistema operativo, ofrecen control fino sobre el intercambio, a costa de exigir que el programador gestione la conexión y el formato de los mensajes.

C. Protocolo TCP

TCP es un protocolo orientado a conexión que garantiza la entrega ordenada y sin pérdidas de los datos mediante confirmaciones, retransmisiones y control de flujo. Estudios sobre redes multi-salto muestran que esta fiabilidad se obtiene a costa de una mayor latencia y sobrecarga frente a protocolos sin conexión [2]. Es la opción natural cuando la corrección de los datos pesa más que la velocidad.

D. Protocolo UDP

UDP es un protocolo sin conexión que envía datagramas independientes sin garantizar entrega, orden ni ausencia de duplicados. A cambio ofrece menor latencia y sobrecarga, lo que lo hace adecuado para tráfico en tiempo real y para escenarios con requisitos de calidad de servicio donde una pérdida ocasional es tolerable [3]. No debe usarse cuando cada mensaje es indispensable.

E. RPC y RMI

La llamada a procedimiento remoto (RPC) abstrae la comunicación de red presentando una invocación remota como si fuera una llamada local, ocultando el empaquetado de parámetros y el transporte subyacente [4]. RMI es su realización orientada a objetos en Java. Internamente, las RPC síncronas se apoyan en un transporte confiable como TCP, por lo que resultan idóneas para invocaciones entre servicios críticos.

F. Mensajería asíncrona

En el paradigma de publicación/suscripción, los emisores publican eventos y los receptores se suscriben a ellos sin conocerse mutuamente, lo que desacopla a las partes en tiempo, espacio y sincronización [5]. Este estilo es la base de la mensajería asíncrona: permite que un servicio continúe su trabajo tras emitir un evento, mientras otros lo procesan a su propio ritmo.

G. Middleware

El middleware es una capa de software que ofrece servicios comunes enrutamiento, seguridad, transformación, entrega garantizada y oculta a las aplicaciones la heterogeneidad de la red y de las plataformas [6]. En el escenario bancario actúan como middleware tanto el API Gateway, que centraliza la entrada de peticiones, como el broker de mensajería, que media en la distribución de eventos.

H. Seguridad y confiabilidad en sistemas bancarios distribuidos

En un banco, dos propiedades son innegociables. La seguridad exige autenticar a las partes y cifrar las comunicaciones sensibles; los protocolos clásicos de autenticación en redes abiertas mostraron cómo el cifrado permite establecer sesiones confiables entre entidades que no se confían de antemano [7]. La confiabilidad transaccional, por su parte, requiere acordar de forma consistente si una operación se confirma o se aborta, incluso ante fallos parciales; este es el problema del commit distribuido, central en el procesamiento de transacciones [8].

IV. MODELADO DE LA COMUNICACIÓN ENTRE PROCESOS

Siguiendo el estilo arquitectónico de microservicios, en el que la aplicación se descompone en servicios pequeños y autónomos que se comunican por la red [9], el sistema bancario se modela mediante los siguientes procesos:

- **Cliente bancario web/móvil:** interfaz del usuario; inicia las operaciones (login, consulta de saldo, transferencia) y recibe las notificaciones.
- **API Gateway:** middleware de entrada; punto único de acceso que enruta las peticiones, aplica TLS y delega en los servicios internos.
- **Servicio de Autenticación:** valida credenciales y emite los tokens de sesión del cliente.
- **Servicio de Cuentas:** consulta y administra los saldos y los movimientos de las cuentas.
- **Servicio de Transferencias:** ejecuta las transacciones entre cuentas y publica los eventos resultantes.
- **Servicio de Notificaciones:** entrega avisos al cliente (push o correo) a partir de los eventos recibidos.
- **Servicio de Auditoría / Antifraude:** consume los eventos de operación para análisis posterior y detección de anomalías.
- **Broker de mensajería (MOM):** desacopla a productores y consumidores de eventos, garantizando su distribución.
- **Base de datos bancaria:** provee la persistencia transaccional de cuentas y movimientos.

La interacción entre estos procesos no es uniforme: las operaciones críticas siguen un patrón síncrono de petición-respuesta sobre conexiones confiables, mientras que la distribución de eventos posteriores a una operación sigue un patrón asíncrono a través del broker.

V. DIAGRAMA DE COMUNICACIÓN

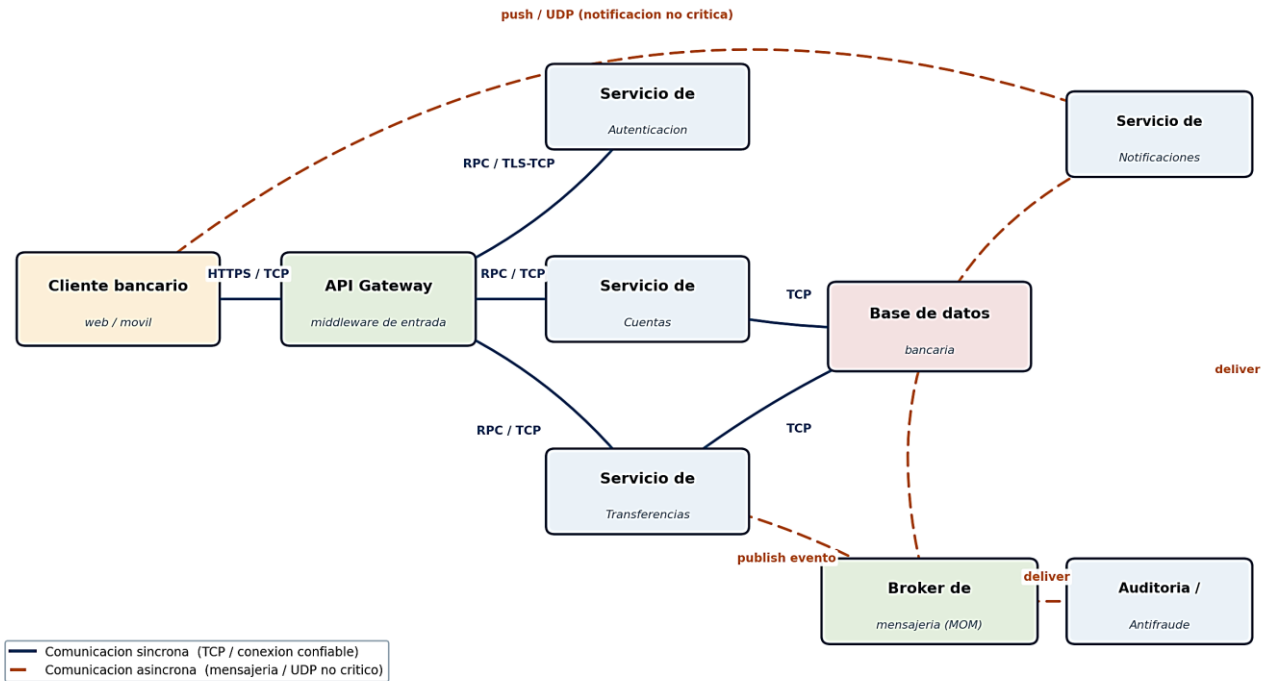


Fig. 1. Diagrama de comunicación entre procesos distribuidos del sistema bancario realizada en draw.io. Las líneas continuas representan comunicación síncrona sobre TCP/conexión confiable; las discontinuas, comunicación asíncrona mediante mensajería o UDP no crítico.

El flujo comienza cuando el cliente envía una petición al API Gateway sobre HTTPS/TCP. El Gateway autentica la sesión contra el Servicio de Autenticación y, según la operación, invoca de forma síncrona al Servicio de Cuentas o al de Transferencias; ambos persisten su estado en la base de datos sobre TCP. Todas estas interacciones son síncronas porque el cliente debe recibir una respuesta correcta antes de continuar.

Tras confirmar una transferencia, el Servicio de Transferencias publica un evento en el broker y responde de inmediato al cliente, sin esperar a sus consecuencias. El broker entrega ese evento, de forma asíncrona, al Servicio de Notificaciones y al de Auditoría/Antifraude. Finalmente, la notificación al cliente un aviso no crítico puede enviarse mediante un push o UDP, donde se prioriza la baja latencia y

se tolera una eventual pérdida. Así, el diagrama separa con claridad las rutas síncronas (continuas) de las asíncronas (discontinuas).

VI. IDENTIFICACIÓN DE TIPO DE COMUNICACIÓN, PROTOCOLO Y MIDDLEWARE

La Tabla I resume, para cada par de procesos que se comunican, el tipo de comunicación, el protocolo de transporte, el middleware involucrado y la justificación de la elección. Las operaciones críticas login, consulta de saldo y transferencia emplean conexiones confiables tipo TCP, mientras que las notificaciones no críticas y la propagación de eventos usan mensajería asíncrona o UDP.

TABLA I
Tipo de comunicación, protocolo y middleware por interacción

Proceso origen	Proceso destino	Tipo de comunicación	Protocolo	Middleware	Justificación
Cliente bancario	API Gateway	Síncrona (petición-respuesta)	HTTPS sobre TCP	API Gateway	El acceso del usuario exige una sesión confiable y cifrada; TCP garantiza entrega ordenada de cada solicitud [2].
API Gateway	Servicio de Autenticación	Síncrona (RPC)	TLS sobre TCP	API Gateway	El login es una operación crítica; requiere confidencialidad e integridad de las credenciales [7].
API Gateway	Servicio de Cuentas	Síncrona (RPC)	TCP	API Gateway	La consulta de saldo necesita una respuesta inmediata y exacta sobre una conexión confiable [1], [4].
API Gateway	Servicio de Transferencias	Síncrona (RPC)	TCP	API Gateway	La transferencia es una transacción crítica que no admite pérdida ni reordenamiento de mensajes [4].
Servicios de Cuentas / Transferencias	Base de datos bancaria	Síncrona	TCP (driver SQL)	Acceso directo	La persistencia de saldos y movimientos exige consistencia transaccional [8].
Servicio de Transferencias	Broker de mensajería	Asíncrona (publish)	AMQP sobre TCP	Broker / MOM	El evento "transferencia realizada" se publica sin bloquear la respuesta al cliente [5], [6].

Broker de mensajería	Servicio de Notificaciones	Asíncrona (subscribe)	AMQP sobre TCP	Broker / MOM	Desacopla la notificación del flujo crítico y tolera reintentos [5].
Broker de mensajería	Servicio de Auditoría / Antifraude	Asíncrona (subscribe)	AMQP sobre TCP	Broker / MOM	El análisis posterior no debe frenar la operación; se procesa de forma diferida [5], [8].
Servicio de Notificaciones	Cliente bancario	Asíncrona (push)	UDP / push no confiable	Push gateway	Aviso no crítico (p. ej. “sesión iniciada”); prioriza la latencia y tolera pérdidas [3].

VII. RELACIÓN CON LA PRÁCTICA GUÍA01-SOCKETS

La práctica guía01-sockets valida experimentalmente el modelo anterior. Se desarrolló en Java 21 sobre un proyecto Maven creado en IntelliJ IDEA, con los paquetes `ec.edu.uteq.distribuidas.tcp` y `ec.edu.uteq.distribuidas.udp`; el servidor TCP escucha en el puerto 9000 y el UDP en el 9001. Cada clase implementada se corresponde con un elemento del sistema bancario modelado.

La clase `ServidorTCP` multihilo, con un pool de hilos que atiende a cada cliente por separado representa a los servicios bancarios críticos (autenticación, cuentas y transferencias) que requieren una conexión confiable y atención concurrente. La clase `ClienteTCP`, que envía comandos y espera respuesta, encarna al cliente bancario que solicita una operación crítica y aguarda su confirmación. Por su parte, `ServidorUDP` y `ClienteUDP` que intercambian datagramas sin mantener estado— modelan los eventos y notificaciones no críticos, donde la simplicidad y la baja latencia priman sobre la garantía de entrega.

VIII. PROCEDIMIENTO REALIZADO

- 1) Creación del proyecto Maven guía01-sockets en IntelliJ IDEA con el arquetipo quickstart y el groupId `ec.edu.uteq.distribuidas`.
- 2) Configuración de Java 21 en el archivo `pom.xml` (source y target 21, codificación UTF-8).
- 3) Creación de los paquetes `tcp` y `udp` dentro del paquete principal.

- 4) Implementación de `ServidorTCP` en el puerto 9000, con pool de hilos y atención de los comandos HORA, ECO y SALIR.
- 5) Implementación de `ClienteTCP`, que se conecta al servidor y envía comandos desde consola.
- 6) Implementación de `ServidorUDP` en el puerto 9001, que recibe datagramas de hasta 1024 bytes y responde con un ACK.
- 7) Implementación de `ClienteUDP`, que envía un datagrama y espera la respuesta con un timeout de 3 segundos.
- 8) Ejecución y verificación: se inician primero los servidores y luego los clientes, comprobando el intercambio en consola.

IX. RESULTADOS

Sobre TCP, el cliente se conectó al puerto 9000 y el servidor respondió correctamente a cada comando: HORA devolvió la hora del servidor (p. ej. `HORA_ACTUAL:16:09:56.194`), ECO Hola mundo devolvió `ECO[#1]:Hola mundo`, y SALIR cerró la conexión con `ADIOS:hasta luego cliente #1`. Estos resultados evidencian una comunicación orientada a conexión, ordenada y confiable, propia de las operaciones bancarias críticas.

Sobre UDP, el cliente envió el mensaje CASA al puerto 9001 y recibió como respuesta el datagrama `ACK: CASA @ 16:12:23.052`. El intercambio confirma una comunicación sin conexión y sin estado entre paquetes, adecuada para avisos no críticos. La diferencia observada entre ambos protocolos respalda empíricamente las decisiones de modelado de la Tabla I.

La evidencia de la ejecución se presenta en las Fig. 2 a 4.

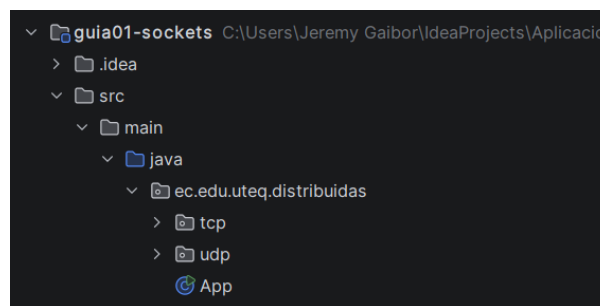


Fig. 2. Estructura del proyecto y paquetes de la práctica (paquetes `tcp` y `udp`).

```

Run: ServidorTCP x
"C:\Users\Jeremy Gaibor\.jdk\ms-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=63:
[16:07:02.366] Servidor listo en :9000

"C:\Users\Jeremy Gaibor\.jdk\ms-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\li
Conectado a localhost:9000
Comandos disponibles: HORA | ECO <texto> | SALIR
-----
> HORA
Servidor: HORA_ACTUAL:16:09:56.194
> ECO Hola mundo
Servidor: ECO[#1]:Hola mundo
> SALIR
Servidor: ADIOS: hasta luego cliente #1
Conexión cerrada.

```

Fig. 3. Ejecución del servidor y el cliente TCP: comandos HORA, ECO y SALIR.

```

Run: ServidorTCP x, ServidorUDP x
"C:\Users\Jeremy Gaibor\.jdk\ms-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\
Servidor UDP escuchando en puerto 9001

"C:\Users\Jeremy Gaibor\.jdk\ms-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\
Cliente UDP. Escribe un mensaje (o 'salir'):
CASA
Respuesta: ACK: CASA @ 16:12:23.052

```

Fig. 4. Ejecución del servidor y el cliente UDP con respuesta ACK.

X. DISCUSIÓN

TCP es adecuado para las transacciones bancarias porque su naturaleza orientada a conexión asegura que cada mensaje llegue una sola vez y en orden; sobre él, las invocaciones RPC entre servicios críticos heredan esa fiabilidad [4]. En cambio, UDP no debe emplearse para transferencias ni para consultas de saldo: la pérdida silenciosa de un datagrama produciría saldos erróneos u operaciones no aplicadas, sin que el sistema lo advierta.

UDP y la mensajería asíncrona sí tienen sentido en el borde no crítico del sistema: notificaciones informativas, telemetría o eventos donde una pérdida ocasional es aceptable y la latencia importa [3], [5]. El middleware es la pieza que hace viable esta separación: el API Gateway concentra seguridad y enrutamiento en la entrada, y el broker de mensajería desacopla la generación de eventos de su consumo [6], de modo que añadir un nuevo consumidor, por ejemplo, un servicio antifraude no obliga a modificar al productor. La experiencia industrial en migraciones a microservicios confirma que este desacoplamiento, bien gobernado, mejora la mantenibilidad y la escalabilidad del sistema [10].

XI. CONCLUSIONES

Conclusión general: el modelado del sistema bancario muestra que la elección del mecanismo de comunicación no es arbitraria, sino que se deduce de los requisitos de cada operación. Las operaciones críticas exigen comunicación síncrona y confiable sobre TCP, mientras que los eventos no

críticos admiten mensajería asíncrona o UDP; la práctica guía01-sockets validó experimentalmente ambos comportamientos.

Freddy Farinango: comprendí que el escenario de negocio es lo que da sentido a la decisión técnica; el mismo socket adquiere un propósito distinto según represente un login o un simple aviso.

Jeremy Gaibor: el diagrama y la tabla me permitieron ver el sistema como un conjunto de interacciones con exigencias diferentes, y no como un único bloque de código cliente-servidor.

Iván Villamarín: relacionar las clases TCP y UDP con servicios bancarios concretos hizo tangible por qué TCP protege una transferencia y por qué UDP basta para una notificación.

ANEXOS

Se adjuntan como evidencia las capturas de la estructura del proyecto (Fig. 2) y de la ejecución de los clientes y servidores TCP y UDP (Fig. 3 y Fig. 4). Enlace para el acceso al repositorio del equipo es:

<https://github.com/JeremyGaibor/Aplicaciones-Distribuidas/tree/main/taller1/guia01-sockets>

Enlace para el acceso al diagrama en draw.io:

https://app.diagrams.net/#G17XeZOYevM_ZYyWDI8FbkSTHiOfxKx5xj#%7B%22pageId%22%3A%221jdJG5vO1JWf2Arma_T4%22%7D

REFERENCIAS

- [1] M. Xue and C. Zhu, “The socket programming and software design for communication based on client/server,” in Proc. 2009 Pacific-Asia Conf. on Circuits, Communications and System (PACCS), Chengdu, China, 2009, pp. 775–777, doi: 10.1109/PACCS.2009.89.
- [2] S. Bansal, R. Shorey, and A. A. Kherani, “Performance of TCP and UDP protocols in multi-hop multi-rate wireless networks,” in Proc. 2004 IEEE Wireless Communications and Networking Conf. (WCNC), Atlanta, GA, USA, 2004, vol. 1, pp. 231–236, doi: 10.1109/WCNC.2004.1311548.
- [3] D. K. Y. Yau and S. S. Lam, “Migrating sockets—End system support for networking with quality of service guarantees,” IEEE/ACM Trans. Netw., vol. 6, no. 6, pp. 700–716, Dec. 1998, doi: 10.1109/90.748083.
- [4] A. D. Birrell and B. J. Nelson, “Implementing remote procedure calls,” ACM Trans. Comput. Syst., vol. 2, no. 1, pp. 39–59, Feb. 1984, doi: 10.1145/2080.357392.
- [5] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, “The many faces of publish/subscribe,” ACM Comput. Surv., vol. 35, no. 2, pp. 114–131, Jun. 2003, doi: 10.1145/857076.857078.
- [6] P. A. Bernstein, “Middleware: a model for distributed system services,” Commun. ACM, vol. 39, no. 2, pp. 86–98, Feb. 1996, doi: 10.1145/230798.230809.
- [7] R. M. Needham and M. D. Schroeder, “Using encryption for authentication in large networks of computers,” Commun. ACM, vol. 21, no. 12, pp. 993–999, Dec. 1978, doi: 10.1145/359657.359659.
- [8] J. Gray and L. Lamport, “Consensus on transaction commit,” ACM Trans. Database Syst., vol. 31, no. 1, pp. 133–160, Mar. 2006, doi: 10.1145/1132863.1132867.
- [9] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: yesterday, today, and tomorrow,” in Present and Ulterior Software Engineering, M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216, doi: 10.1007/978-3-319-67425-4_12.
- [10] J. Fritzsche, J. Bogner, S. Wagner, and A. Zimmermann, “Microservices migration in industry: intentions, strategies, and challenges,” in Proc. 2019 IEEE Int. Conf. on Software Maintenance and Evolution (ICSME), Cleveland, OH, USA, 2019, pp. 481–490, doi: 10.1109/ICSME.2019.00081.